

Chapter 1

Path Config Documentation

1.1 Introduction

The `path_config.py` module provides centralized path configuration for CTCC scripts. It defines absolute paths based on the location of `path_config.py` (inside `scripts/`) so that YAML config and output directories are resolved consistently regardless of the current working directory. There are no environment overrides; paths are derived from `Path(__file__).parent.absolute()`.

1.2 Method Overview

1.2.1 Purpose

1. Define `SCRIPT_DIR` as the directory containing `path_config.py` (i.e. `scripts/`).
2. Define `PROJECT_ROOT` as the parent of `SCRIPT_DIR` (the CTCC project root).
3. Define `YAMLS_DIR = PROJECT_ROOT / "yamls"` (YAML configuration files).
4. Define `OUTPUTS_DIR = PROJECT_ROOT / "outputs"` (CSV and JSON output files).
5. Define `SCRIPTS_DIR = SCRIPT_DIR` (same as `scripts/`).
6. Export `YAMLS_DIR`, `OUTPUTS_DIR`, `PROJECT_ROOT`, `SCRIPTS_DIR` via `__all__`.

1.2.2 Calculation Approach

Paths only; no cost/benefit logic. All paths are `pathlib.Path` instances. Base directory is the script location, not the current working directory, so running `python scripts/build_costs.py` from the project root or from elsewhere yields the same `YAMLS_DIR` and `OUTPUTS_DIR` relative to the project.

1.2.3 Inputs and Outputs

Inputs:

- None (no env vars or function arguments); paths are computed from `__file__`.

Outputs:

- Module-level constants: `YAMLS_DIR`, `OUTPUTS_DIR`, `PROJECT_ROOT`, `SCRIPTS_DIR`. Used by `yaml_loaders`, `weighted_miles`, `build_costs`, `row_costs`, `insurance_costs`, `environmental_mitigation`, `oandm`, `energy_losses`, `bcr_calculator`, `csv_output_manager`, `smart_loaders`, `revenue`.

1.2.4 Integration with Other Scripts

- `yaml_loaders.py` — Uses `YAMLS_DIR` for all YAML file paths (e.g. `YAMLS_DIR / "01_project_technical_details.yaml"`).
- `weighted_miles.py` — `YAMLS_DIR / "02_project_physical_details.yaml"`.
- `build_costs.py`, `row_costs.py`, `insurance_costs.py`, `environmental_mitigation.py`, `oandm.py`, `energy_losses.py` — `YAMLS_DIR` for config YAMLs.
- `bcr_calculator.py` — `OUTPUTS_DIR` for `batch_summary.csv`, `YAMLS_DIR` for BCR config YAML.
- `csv_output_manager.py` — `OUTPUTS_DIR` (default output dir), `YAMLS_DIR` where used.
- `smart_loaders.py` — `YAMLS_DIR` when in YAML mode for `get_project_data_raw`, `get_financing_data_raw`, `get_physical_data_raw`.
- `revenue.py` — `OUTPUTS_DIR` for `batch_summary.csv` and JSON output paths.

1.3 Key Functions

There are no callable functions; only module-level constants are defined.

1.3.1 Constants

Key logic (path resolution):

```
1 SCRIPT_DIR = Path(__file__).parent.absolute()
2 PROJECT_ROOT = SCRIPT_DIR.parent
3 YAMLS_DIR = PROJECT_ROOT / "yamls"
4 OUTPUTS_DIR = PROJECT_ROOT / "outputs"
5 SCRIPTS_DIR = SCRIPT_DIR
6 __all__ = ["YAMLS_DIR", "OUTPUTS_DIR", "PROJECT_ROOT", "SCRIPTS_DIR"]
```

- `SCRIPT_DIR` — Absolute path to `scripts/` (directory containing `path_config.py`).

- `PROJECT_ROOT` — Absolute path to the CTCC project root (parent of `scripts/`).
- `YAMLS_DIR` — `PROJECT_ROOT / "yamls"`; used for all YAML config file paths.
- `OUTPUTS_DIR` — `PROJECT_ROOT / "outputs"`; used for `batch_summary.csv`, `json_output_*.json`, `ctcc_results_*.json`.
- `SCRIPTS_DIR` — Same as `SCRIPT_DIR`; alias for the scripts directory.

1.4 Example

Typical usage in a script:

```
1 from path_config import YAMLS_DIR, OUTPUTS_DIR
2 with open(YAMLS_DIR / "02_project_physical_details.yaml", "r") as f:
3     data = yaml.safe_load(f)
4 # ...
5 output_path = OUTPUTS_DIR / "batch_summary.csv"
```

If `path_config.py` is at `/path/to/CTCC/scripts/path_config.py`, then `PROJECT_ROOT` is `/path/to/CTCC`, `YAMLS_DIR` is `/path/to/CTCC/yamls`, and `OUTPUTS_DIR` is `/path/to/CTCC/outputs`, regardless of `os.getcwd()`.